Deliverable 7

Group Name: The Reservists
System Name: Carrel
Members: Kevin Wang, Chris Wu, Rui Lin

Part 1: Software Project Management

Requirements Engineering as a Success Factor in Software Projects

Authors: Hubert F. Hofmann and Franz Lehner

Summary

Hofmann and Lehner examine fifteen requirements engineering teams to explain why some projects benefit from RE while others still suffer from rework and delay. Their central argument is that RE is not a one-time documentation phase. It is a continuing process of learning, communication, negotiation, and validation. Its effectiveness depends on the balance among team knowledge, available resources, and the process used to discover and manage requirements.

Successful teams combine domain knowledge, technical knowledge, and knowledge of RE practices. Because no individual is likely to possess all three, teams need complementary expertise and direct access to users and domain specialists. Relying only on intermediaries can distort stakeholder needs. Training and capable project leadership are also important because subject knowledge alone does not guarantee effective prioritization, modeling, or validation.

Requirements work also needs visible time and effort. Under delivery pressure, teams may reduce reviews and customer interaction because these activities do not immediately produce code. The study suggests that this creates more rework later. Strong teams use templates and examples to build a shared language, communicate frequently, and treat requirements as living agreements. They prioritize explicitly, combine models with prototypes, trace important requirements, and validate repeatedly through reviews, walkthroughs, scenarios, and stakeholder feedback. The best results

come from adjusting knowledge, resources, and process together as project uncertainty changes.

Application to Real-World Software Project Management

In projects with frequent change, managers should replace an unrealistic requirements freeze with controlled change management. Each significant change should record its source, rationale, priority, owner, and effects on design, schedule, tests, and dependent work. A lightweight traceability matrix makes these effects visible and prevents silent scope growth. Stakeholder conflict should be addressed early through explicit decision criteria and shared scenarios or prototypes. These artifacts make disagreements concrete and help the team decide which needs provide the greatest value.

Managers should also protect a small set of high-value RE activities during schedule pressure, such as weekly requirements triage, peer review of high-risk requirements, and acceptance-criteria review before implementation. Project monitoring should include more than feature completion. It should ask whether critical stakeholders are represented, assumptions are visible, priorities have owners, non-functional requirements are measurable, and recent changes are connected to implementation and verification. These checks expose uncertainty before it becomes expensive rework.

Team Collaboration and Communication Reflection

In a general team project, we would begin with a shared glossary and a consistent requirement format containing the need, rationale, priority, owner, and acceptance condition. This reduces differences in interpretation and prevents knowledge from remaining with the person who first heard a request. Brief recurring synchronization meetings would focus on changed assumptions, blocked decisions, and cross-task dependencies, while a decision log would support members working asynchronously.

We would also review important requirements through examples, sketches, scenarios, or prototypes rather than discussing wording alone. Reviews should ask whether different roles interpret the requirement consistently and whether its acceptance can be demonstrated. Traceability and retrospectives would then help us reconstruct why decisions were made and identify where information was lost. Collaboration improves when knowledge is shareable, responsibility is visible, and feedback arrives early enough to change the work.

The Influence of Human Aspects on Requirements Engineering-related Activities: Software Practitioners' Perspective

Authors: Dulaji Hidellaarachchi, John Grundy, Rashina Hoda, and Ingo Mueller

Summary

Hidellaarachchi, Grundy, Hoda, and Mueller study the human conditions that influence requirements engineering through a survey of 111 practitioners. The results challenge the tendency to focus only on methods and tools. Practitioners identify motivation, domain knowledge, attitude, communication skills, personality, and teamwork as major influences on RE effectiveness. Emotions, cultural diversity, and geographical distribution also matter, while age and gender are considered much less important.

Motivation has both social and technical sources. People are motivated by collaboration, stakeholder engagement, personal interest, job satisfaction, customer value, clear requirements, adequate resources, and confidence that their analysis will affect the outcome. Deadlines or complex work may create a useful challenge, but they become demotivating when expectations are unclear or support is missing. Managers therefore cannot create motivation through pressure or rewards alone; teams also need access to information, stakeholders, and meaningful decisions.

Effective requirements work benefits from complementary personal strengths. Openness and flexibility help analysts explore unfamiliar problems, conscientiousness supports detailed analysis, and empathy and communication support negotiation across roles. The article does not propose one ideal personality. Instead, it suggests that team composition and role fit matter. Reported barriers—including poor communication, unclear requirements, weak domain knowledge, insufficient resources, management problems, difficult stakeholder relationships, uncontrolled changes, and unsuitable methods—are often project conditions rather than individual failures. Performance should therefore consider the clarity and correctness of outputs, responsiveness to feedback, stakeholder interaction, and the context in which the work occurred.

Application to Real-World Software Project Management

Distributed teams need deliberately designed communication. Managers should define where decisions, questions, and urgent issues are recorded; document meeting outcomes; rotate inconvenient meeting times when appropriate; and confirm understanding instead of treating silence as agreement. Ambiguous or conflict-heavy topics benefit from richer interaction, such as a shared modeling or prototype session, while routine updates can remain asynchronous.

Knowledge gaps can be reduced through onboarding, glossaries, examples, pairing, and short domain demonstrations. Roles can reflect complementary strengths—for example, one member may facilitate stakeholder discussions while another refines acceptance criteria or examines feasibility—but pairing and rotation should prevent permanent silos. Performance management should distinguish an individual skill gap from a system constraint. Missing stakeholder access requires a different response from missing training, and counting requirements or tickets rewards volume rather than understanding.

Team Collaboration and Communication Reflection

We would treat knowledge sharing as scheduled project work. Members would briefly explain assumptions, unfamiliar terms, and dependencies, while a glossary, decision log, accepted examples, and open-question list would preserve shared context. Facilitation and presentation roles would rotate so that communication did not depend on one confident member and everyone gained experience translating across perspectives.

Psychological safety would also be essential. We would normalize admitting uncertainty, questioning an assumption, and requesting feedback. Reviews would focus on the artifact and shared goal rather than the status of its author, and facilitators would invite quieter participants to contribute. Fast peer feedback would correct misunderstandings during the work, while retrospectives would examine whether information reached the right people, workloads were realistic, and any team behavior discouraged questions. One or two concrete improvements would then be tested in the next iteration.

Part 2: Requirements Engineering for AI Systems

Requirements Engineering for Artificial Intelligence Systems: A Systematic Mapping Study

Authors: Khlood Ahmad, Mohamed Abdelrazek, Chetan Arora, Muneera Bano, and John Grundy

Summary

Ahmad, Abdelrazek, Arora, Bano, and Grundy review 43 studies to map the emerging field of requirements engineering for AI systems. Traditional RE assumes that system behavior can be specified and checked with reasonable stability. AI components challenge this assumption because their behavior depends on training data, statistical learning, and operating context. Outputs may remain uncertain or difficult to explain even when the surrounding software works as intended.

AI development introduces requirements concerns that conventional processes often leave implicit. Data must be collected, cleaned, labeled, governed, and shown to represent the intended population and context. Models must be selected, trained, and evaluated with measures that may conflict. A model can satisfy an average accuracy target while failing an important subgroup or rare scenario. Requirements must therefore connect stakeholder goals to data characteristics, model behavior, uncertainty, and conditions of use.

The reviewed literature frequently uses UML and goal-oriented RE, with growing interest in domain-specific modeling, but no complete standard method has emerged. Major gaps include data requirements, explainability, trust, fairness, ethics, uncertainty, and verification of predictions. These qualities interact: a complex model may improve performance but reduce interpretability, while stronger data collection may create privacy concerns. The study also identifies weak integration between software engineers and data scientists. AI projects need shared processes that connect software architecture and verification with experimental model development and stakeholder values.

Three Inferred Lessons

1. Data should be treated as a first-class requirements artifact with an owner and acceptance criteria. Because the article connects AI behavior to data suitability, data cannot remain an informal input controlled only by the modeling team. A project should define provenance, coverage, labeling quality, freshness, privacy, permitted uses, known exclusions, and approval responsibility. These conditions should be traced to model evaluation and monitoring. This is an inferred governance response to the article's identified data-requirements gap.

2. Collaboration between software engineers and data scientists must be built into project governance. The article reports an integration gap, which implies that occasional informal coordination is insufficient. Joint reviews should cover stakeholder goals, data assumptions, model metrics, software interfaces, failure handling, and release evidence. Changes to data, models, or operating context should trigger cross-functional impact review. This process preserves experimentation while making system-level consequences and responsibility visible.

3. AI non-functional qualities require continuous monitoring rather than one-time specification. Explainability, fairness, trust, robustness, and ethical acceptability depend on changing data and context. Each important quality should therefore include an initial acceptance threshold and an operational plan defining what will be observed, for whom, how often, by whom, and what triggers escalation, rollback, or retraining. This lesson extends the article's lifecycle challenges into a practical control mechanism.

Formal Requirements Engineering and Large Language Models: A Two-Way Roadmap

Authors: Alessio Ferrari and Paola Spoletini

Summary

Ferrari and Spoletini describe a two-way relationship between formal requirements engineering and large language models. In one direction, formal methods can check requirements artifacts generated or analyzed by LLMs. In the other, LLMs can make

formal RE more accessible by helping people create, translate, query, and understand formal representations. The approaches have complementary strengths: LLMs are fluent and easy to use but may hallucinate or contradict themselves, while formal methods provide precise, machine-checkable reasoning but require specialized expertise.

Formal analysis can separate a plausible proposal from evidence. LLM-produced requirements may be translated into logic or models and checked for consistency, satisfiability, safety, security, fairness, or domain constraints. This is valuable because fluent wording can hide fabricated assumptions or contradictions. Formal verification still cannot prove that the model reflects stakeholders' true intentions, so human validation remains necessary.

In the opposite direction, LLMs may translate natural-language requirements into candidate formal artifacts, explain formulas in domain language, describe counterexamples, and help users navigate formal tools. This could reduce cost and broaden participation, but generated artifacts and explanations must themselves be checked. The roadmap therefore supports a combination of automation, rigor, and human judgment rather than replacement of analysts. It is relevant to safety-critical systems and to ordinary services with important privacy, security, or fairness requirements.

Three Inferred Lessons

1. The most practical process is hybrid collaboration among humans, an LLM, and formal verification. Humans define goals, context, risk tolerance, and final acceptance; the LLM proposes or translates artifacts; and a formal engine checks selected properties. Humans then judge whether the formal assumptions match the intended problem. This division of labor is inferred by combining the roadmap's two directions and allows speed without transferring accountability to the model.

2. Natural-language explanation of formal artifacts is adoption infrastructure, not optional convenience. Stakeholders need to understand what a property means, which assumptions it uses, why a counterexample matters, and what change might resolve it. Explanations should remain linked to the underlying formula, support different levels of detail, and allow expert correction. This inference turns the article's explanation capability into an organizational bridge between proof-oriented tools and decision makers.

3. The level of formalization should scale with project risk. Low-risk features may benefit from structured terminology and simple consistency checks; medium-risk work may formalize important business rules and scenarios; safety- or rights-critical functions may require comprehensive models, independent review, and traceable evidence. This risk-based ladder is inferred from the roadmap's broad relevance and makes formal RE practical when an organization cannot formalize every requirement.